

Q1. Using the same preprocessing method as the Counting Sort sorting algorithm, the algorithm in question would have a preprocessing time of $\Theta(n + k)$ when given n integers in range 0 to k to sort. This preprocessing section would have the array carrying the integers input as $A[1..n]$ and would create two new arrays, where $B[1..n]$ would hold the stored output and array $C[0..k]$ would act as a working storage. Iterating through array C , setting all values to 0 then iterating through array A , inspecting each input element and if the value of the input is the same as the position number in the C array, then the input at that position is incremented in the C array. This leaves array C with the number of input elements equal to the position value for all integers between 0 to k . Then iterate through C to determine how many input elements are less than or equal to the position value in C .

After the pre-processing with time complexity of $\Theta(n + k)$, the algorithm would be able to check how many integers are within a range of $[a..b]$ but calculating $C[b] - C[a-1]$. This would show the number of integers (represented in the C array) that are less than or equal to b and greater than or equal to a as these values are used as the positions in the C array. As this is a calculation, this would take time $O(1)$.

Proof:

Base Case: When the array used is empty or of size 1

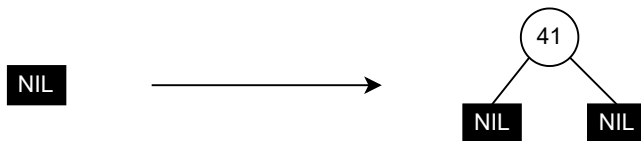
- Array A will be either empty or contain 1 element
- Array C will be either empty or contain 1 zero element
- Then the calculations for determining how many integers are within the range of $[a..b]$ will only be $[0]$ or $[0, 1]$ which would simply output the one value in the C array (if the array contains one element)
- This whole process would take $O(1)$ time
- After the pre-processing, the only value that can be used to determine how many values are within a range, is the only element in C array so this will be returned in $O(1)$ time.

Inductive case: When the array used has more than 1 element (n elements)

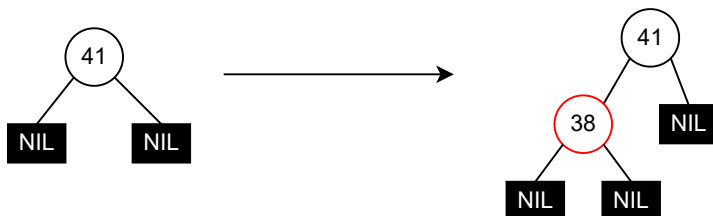
- Array A will contain n elements, as will array C (but of 0 value)
- Iteration through array A to determine if the value at the current position is equal to the position value in C should take $O(n)$ time as it goes through every element
- Iteration through C such that for every element it is changed $C[i] + C[i-1]$ over the course of 1 through k integers (to indicate the number of elements less than or equal to i) should take around $O(k)$ time
- Together this would leave the preprocessing as $O(n+k)$
- When performing the calculation to determine how many integers are within a range of $[a..b]$, if you take the element at $C[a-1]$ and subtract it from the element at $C[b]$ then this would give the cumulative amount of variables within that range. This basic calculation would take $O(1)$ time.

This proves the correctness of this algorithm and the fact that the preprocessing should run in $\Theta(n + k)$ and the final query would run on $O(1)$ time.

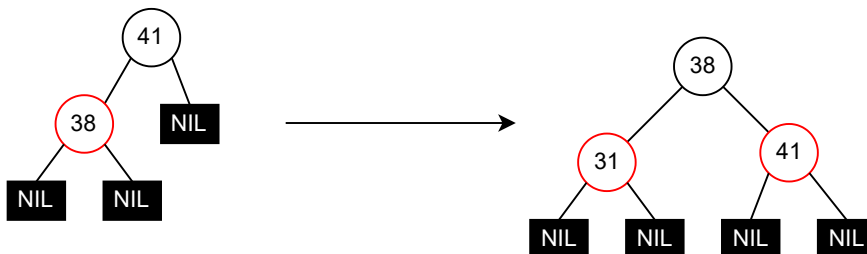
Q2. Insertion: 41



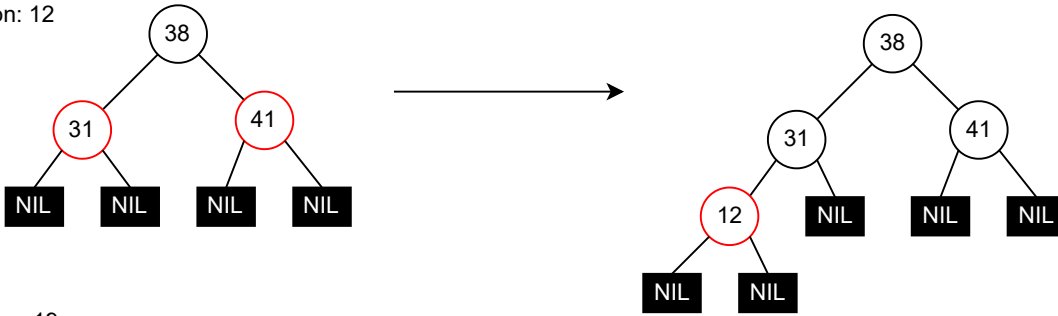
Insertion: 38



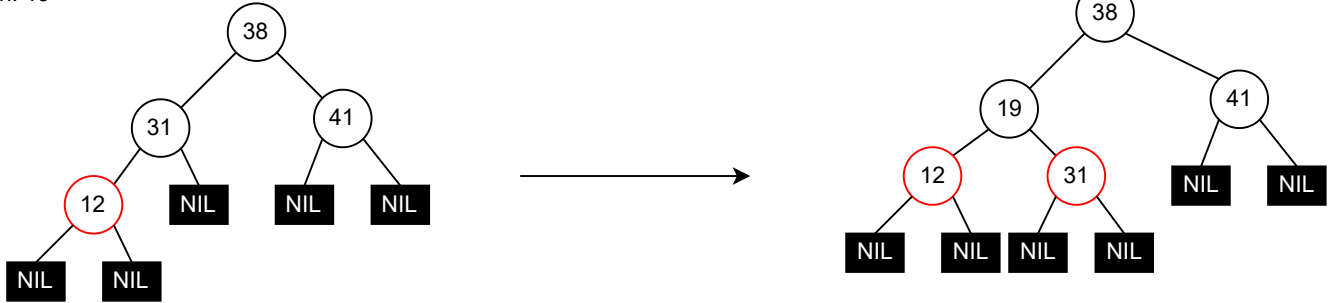
Insertion: 31



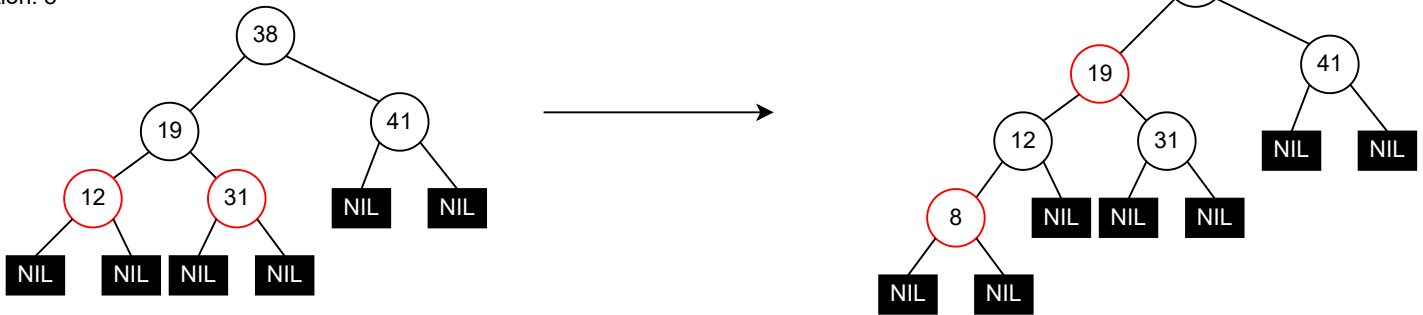
Insertion: 12



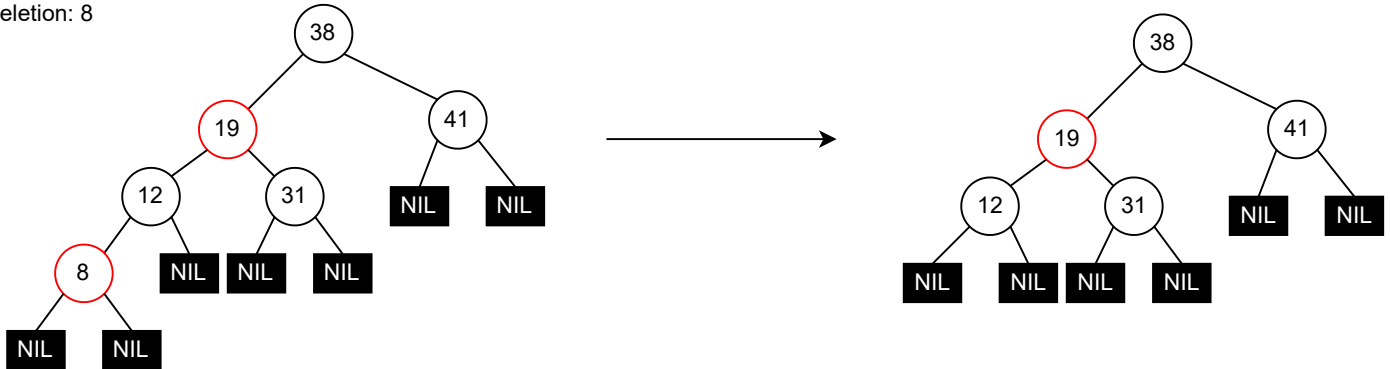
Insertion: 19



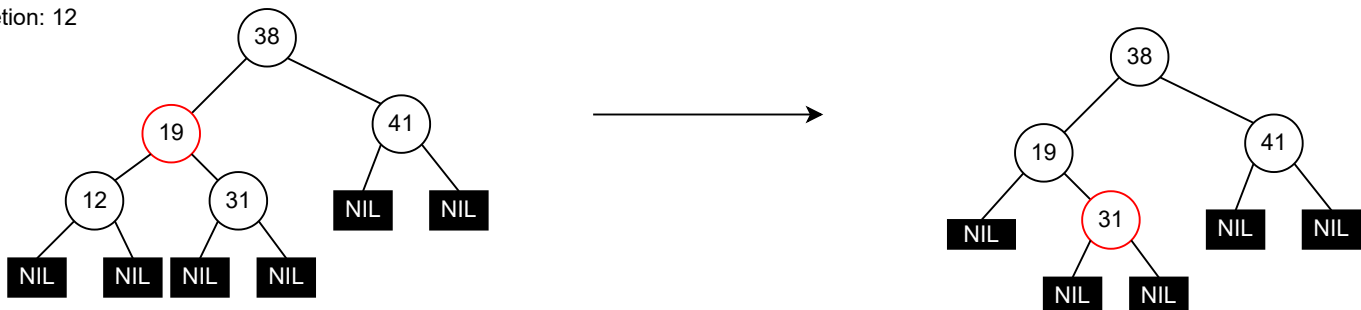
Insertion: 8



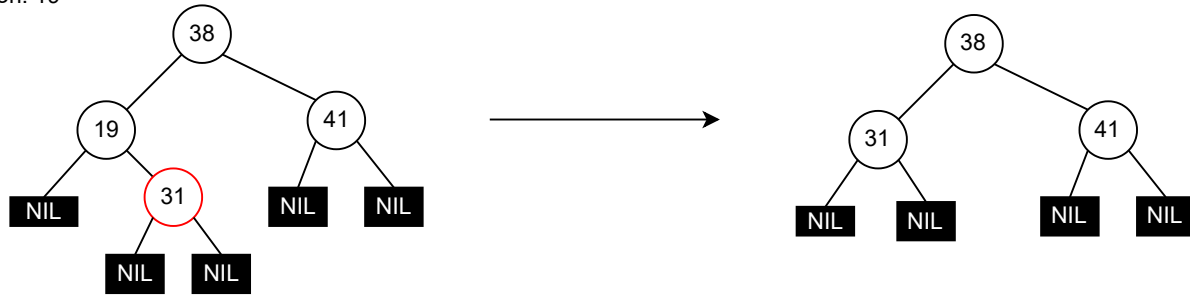
Q3. Deletion: 8



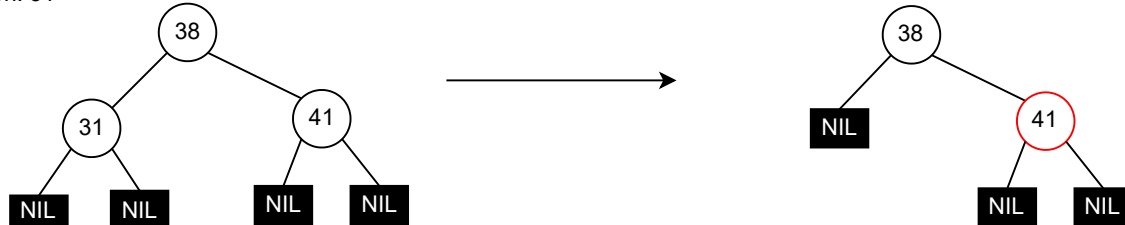
Deletion: 12



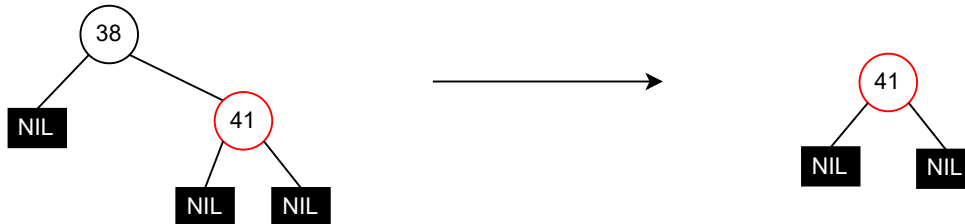
Deletion: 19



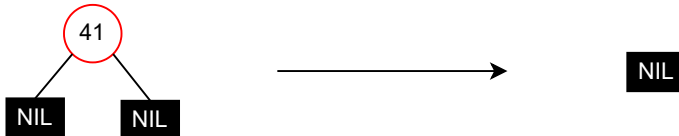
Deletion: 31



Deletion: 38



Deletion: 41



Q4. Suppose that a node x is inserted into a red-black tree with RB-insert and then is immediately deleted with RB-delete. Is the resulting tree the same as the initial red-black tree?

If a node x is added to a red-black tree and then immediately deleted, it does not mean that the resulting tree would be the same as the initial tree. If the tree was initially empty and node x was added and then deleted then the resulting tree would always be the same as the initial tree, however, this could change if there is at least two nodes in the tree. If a tree was organized such that the numbers 10, 25, 46 were added to a red black tree, when 10 is added it is a black root node. When 25 is added it goes to the right child of 10 and is colored red. However, no issues will be brought up if you need to delete a red node, because the remaining tree will still be a proper red-black tree. So another node, 46 can be added to the right of node 25 and is colored red. This would lead to a change in the other two nodes of the tree as a red node (25) cannot have a red child node (26). This would lead to a right rotation of the tree leaving a black 25 node as the root with 10 and 46 as its red children nodes. Now if 46, which in this example is node x , is deleted, then the resulting tree is now different than what it was, prior to node 46's insertion.

Q5. Prove that an AVL tree with n nodes has height $O(\log n)$. Proof by induction:

To do this we prove the fact that an AVL of height h has at least F_h nodes, where F_h is the h^{th} Fibonacci number.

Base case: When $h = 0$

- This would mean that the Fibonacci number would be 0, meaning that there is 1 node in the AVL tree, the root node and since $1 > 0$, this holds true.

Base case: $h = 1$ and $h = 2$

- This would mean that the Fibonacci number would be 1, meaning that the tree would have to have a minimum of 1 node. This holds true seeing as height of 1 means that there is at least a root node with a child node and a height of 2 means that there is at least a root node, a child on both ends, and a child of one of the children nodes.
- In both cases $n > 2$ is greater than 1, so this holds true

Inductive case: for $h > 2$, the tree would have to contain $F_h = F_{h-1} + F_{h-2}$ nodes

- For example, for $h = 3$ for example, then the Fibonacci number would be $F_3 = 1 + 1 = 2$ and there would have to be a minimum of a root node, a child node, a child of that child, and a child of that child for the tree height to be 3.
- Since this is an AVL tree. one of the subtrees of the tree would have to have at least a height of $h - 1$ while the other has a height of $h - 2$ to hold the AVL tree property.
- This means one subtree (the children nodes and downwards from the root node) would have at least $F_{h-1} = F_{h-2} + F_{h-3}$ nodes (n) and the other would have at least $F_{h-2} = F_{h-3} + F_{h-4}$ nodes (n)
- This successive division of the subtrees for the AVL tree leads to notion that the height of the AVL tree is $O(\log n)$

Q6. Given the fact that all k sequences are already sorted, an algorithm to sort all kn elements could use a heap sort technique. This algorithm would have to check the position value i starting from 0 to n , of each of the k sequences at the i^{th} position and insert that value in the heap. This would be done by creating a heap of size k , then inputting all the values from each of the k sequences at the first position into the heap. Each value added from the k sequences would be inserted such that the largest of these values would be swapped with any values so that the smallest values get pushed down the tree. When the initial heap is done being made, the smallest value can be removed from the tree and input into the first position of the final sorted array (of size kn). this value would have to be the smallest value of all the kn elements as the k sequences are already sorted so the smallest value should be in the first position of one of the k sequences. After this value is added to the final array, the next value in that respective sequence ($i+1$ position of that sequence) would be inserted into the heap in place of the one that was removed. The heap property would have to be maintained by moving the largest values, swapping it with the array at its respective position. After this addition to the heap, the smallest value from this would be taken and put into the array that is to be returned. This process would continue until all kn elements are put into the array to be returned. When the algorithm reaches the n value for each of the k sequences, it will take the smallest values from the heap in succession until the heap is empty.

This algorithm should have a time complexity of $O(kn \log k)$, which is a combination of the heapsort worst case time complexity, $O(k \log k)$ where k is the number of elements that would need to be in the heap and the fact that it needs to be done n times for each sequence.

The correctness of this algorithm:

Base Case: When there is one sequence ($k = 1$) with only one element long ($n = 1$)

- When adding 1 element from each sequence to the heap, the first element added will not violate heap rules and can be added as is.

Inductive Case(1): When there are multiple sequences ($k > 1$) but each sequence is only 1 element long ($n = 1$)

- When adding each of the single elements to the heap, if a violation of the heap occurs at position i (of k) then a swap will occur, else the algorithm would have to explore $k/2$ sections of the heap. This would leave a time complexity of $O(\log k)$.
- The smallest element of the heap would be located and removed in $O(1)$ time and put into the organized array to be returned. This would be repeated for each resulting heap until it is empty leaving an organized array of the sequence elements.

Inductive Case (2): When there are multiple sequences ($k > 1$) and each sequence is many elements long ($n > 1$)

- When adding each of the single elements to the heap, if a violation of the heap occurs at position i (of k) then a swap will occur, else the algorithm would have to explore $k/2$ sections of the heap. This would leave a time complexity of $O(\log k)$.
- This would need to happen for every single element (n) of every single sequence (k) as, each time the smallest value in the heap is removed to be placed in the ordered array to be returned, the value would need to be replaced by the next element in its respective sequence.
- This would leave a complexity of $(kn \log k)$, multiplying the complexity of maintaining a heap by the number of elements that are going to be added to it.

Q7. Knowing that the rank of a node goes up one value if it is a union with another node of the same rank value, and if not its rank value becomes the larger of the two node ranks, and the sequence of the operations $\text{Make_set}(i)$ and $\text{Union}(i)$, a node having a rank of at most $\lceil \log n \rceil$ can be proven. Using induction of the sequence $\text{Make_set}(i)$:

Base case: When the $\text{Make_set}(1)$ is called, it creates a node with a rank of 0

Inductive step: $\text{Union}(i, j)$ operation joining the roots containing x_i and x_j .

- As only each root node of a set is compared by the $\text{Union}(i, j)$ operation, for a sequence of m operations, n of them are Make_set , the tree height is less than or equal to $\log n$
- Since we know that for any node, its height is bound by its rank, then that means the every node at that height must have at most $\lceil \log n \rceil$ rank simply so that they would be able to be within the same height level.

Q8. In light of the previous question, $x.rank$ would need $O(\log \log n)$ bits for each node, so it would be possible to store rank in 1 byte as the value is asymptotically negligible.

Q9. No compression scheme can expect to compress a file of randomly chosen 8-bit characters by even a single bit, because if a Huffman code were applied to a sequence of letters that are all equally likely to occur, if the ASCII letters are represented using 8-bits, then there would be no compression, or difference from if no compression was applied. Because there would be no change in file size or representation of the characters then even when the Huffman code compression scheme is applied, the file size would not be reduced.

Q10a. The `final_sets()` algorithm creates an array of `Set<Integer>` objects big enough to hold the current amount of sets stored. This will leave `make_set(i)` and `union_sets(i, j)` functions useless, as even if they are made and bound, they will be added to a list that is not going to be used. The Representative for each set which is made in the `Rep<Integer>` class and points to the head of a given set, has a `setRep(Set<T>)` function that allows you to change the head of the set. A for loop iterating from 0 to the stored size (list length) will be used to make a new `Set/node` that will have that value as its element, then this node will have its `setNext(Set<T>)` function called and sets its next node to the current head of the set. Then the representative node will have its `setRep(Set<T>)` function called, setting the new head to the node that contains the correct representation value. The function will then return the size of the list.